

32 – POINT FAST FOURIER TRANSFORM USING VERILOG

by

S SHANMATI	22BEC1010
SANJANA A	22BEC1013
V B VARSHINI	22BEC1152

A project report submitted to

Dr. VELMATHI G

SCHOOL OF ELECTRONICS ENGINEERING

in partial fulfilment of the requirements for the course of

BECE303L – VLSI SYSTEM DESIGN

in

**B.Tech. ELECTRONICS AND COMMUNICATION
ENGINEERING**



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

Vandalur – Kelambakkam Road

Chennai – 600127

NOVEMBER 2024

BONAFIDE CERTIFICATE

Certified that this project report entitled “**32 – POINT FAST FOURIER TRASNFORM USING VERILOG** ” is a bonafide work of **S SHANMATI – 22BEC1010, SANJANA A – 22BEC1013, V B VARSHINI – 22BEC1152** who carried out the Project work under my supervision and guidance for **BECE303L – VLSI SYSTEM DESIGN**.

Dr. VELMATHI G

Professor - Higher Academic Grade

School of Electronics Engineering (SENSE),

VIT University, Chennai

Chennai – 600 127.

ABSTRACT

The Fast Fourier Transform (FFT) is an essential algorithm in digital signal processing (DSP), enabling the conversion of time-domain signals into their frequency-domain representations with reduced computational complexity. This project presents the design and implementation of a 32-point FFT using Verilog, based on the Decimation-in-Time (DIT) butterfly algorithm. The objective of this work is to develop an efficient and scalable hardware model for FFT that can be used in real-time signal processing applications, such as communications and audio/video processing.

The 32-point FFT is divided into multiple stages, where each stage processes the input signal by applying the butterfly operation, which combines two smaller FFTs into one larger FFT. The project focuses on optimizing the design by using fixed-point arithmetic for efficient resource usage, ensuring both accuracy and minimal hardware overhead. A key feature of the design is the use of parallelism and pipelining to achieve high throughput and low latency, allowing for faster computation of FFT results.

Verilog is used as the hardware description language to model the FFT algorithm, and the design is tested through extensive simulations to ensure correctness and functionality. The performance of the implementation is evaluated in terms of resource usage, speed, and accuracy, with results demonstrating that the design efficiently performs the 32-point FFT within the constraints of the chosen hardware. This project highlights the potential for Verilog-based FFT designs in embedded systems and signal processing applications where computational efficiency and speed are critical.

ACKNOWLEDGEMENT

We wish to express our sincere thanks and deep sense of gratitude to our project guide, **Dr. VELMATHI G**, Professor - Higher Academic Grade, School of Electronics Engineering, for his consistent encouragement and valuable guidance offered to us in a pleasant manner throughout the course of the project work.

We are extremely grateful to **Dr. Ravi Sankar A**, Dean of School of Electronics Engineering, VIT Chennai, for extending the facilities of the School towards our project and for her unstinting support.

We also take this opportunity to thank all the faculty of the School for their support and their wisdom imparted to us throughout the course.

We thank our parents, family, and friends for bearing with us throughout the course of our project and for the opportunity they provided us in undergoing this course in such a prestigious institution.

TABLE OF CONTENTS

Chapter	Title		Pg. No.
	Bonafide Certificate		2
	Abstract		3
	Acknowledgement		4
	Table of Contents		5
1.	Introduction		6-7
	1.1	Objectives	
	1.2	Benefits	
	1.3	Features	
2.	Proposed System		8-16
	2.1	Block Diagram	
	2.2	Butterfly Unit	
	2.3	Address Generation Unit	
	2.4	Twiddle factor ROM	
	2.5	Dual Port RAM	
	2.6	MUX	
3.	Fast Fourier Transform		17-22
	3.1	Verilog Code for 8-Point FFT and RTL	
	3.2	Verilog Code for 32-Point FFT	
	3.3	Results	
	3.4	RTL	
	3.5	Inferences	
4.	Conclusion		23
5.	References		24
6.	Biodata		25

CHAPTER 1

INTRODUCTION

1.1 OBJECTIVES

- **Design a 32-Point FFT Model:** Develop a functional 32-point FFT architecture using the Decimation-in-Time (DIT) butterfly algorithm and implement it using Verilog.
- **Optimize for Performance:** Ensure the FFT implementation achieves low latency and high throughput through parallelism and pipelining techniques.
- **Validate Accuracy and Functionality:** Perform rigorous simulations to validate the correctness, stability, and performance of the FFT output.
- **Minimize Hardware Utilization:** Implement resource-efficient fixed-point arithmetic to ensure the design uses minimal hardware resources without compromising performance.
- **Provide Application-Ready Design:** Create a robust FFT model suitable for integration into larger DSP systems, making it versatile for applications requiring frequency analysis.

1.2 BENEFITS

- **Enhanced Computational Efficiency:** The 32-point FFT reduces the computational complexity of signal processing by transforming signals from the time domain to the frequency domain more efficiently than traditional Discrete Fourier Transform (DFT) methods.
- **Real-Time Processing Capability:** Due to its high speed and parallel processing structure, the design enables real-time processing, essential for applications in communications and digital media.
- **Resource Optimization:** By using fixed-point arithmetic and efficient hardware structuring, the Verilog-based design optimizes resource usage, making it suitable for hardware with limited capacity.

- **Scalability for Larger Transforms:** The modular design can serve as a building block for larger FFT sizes, making it flexible for various applications in signal processing and telecommunications.

1.3 FEATURES

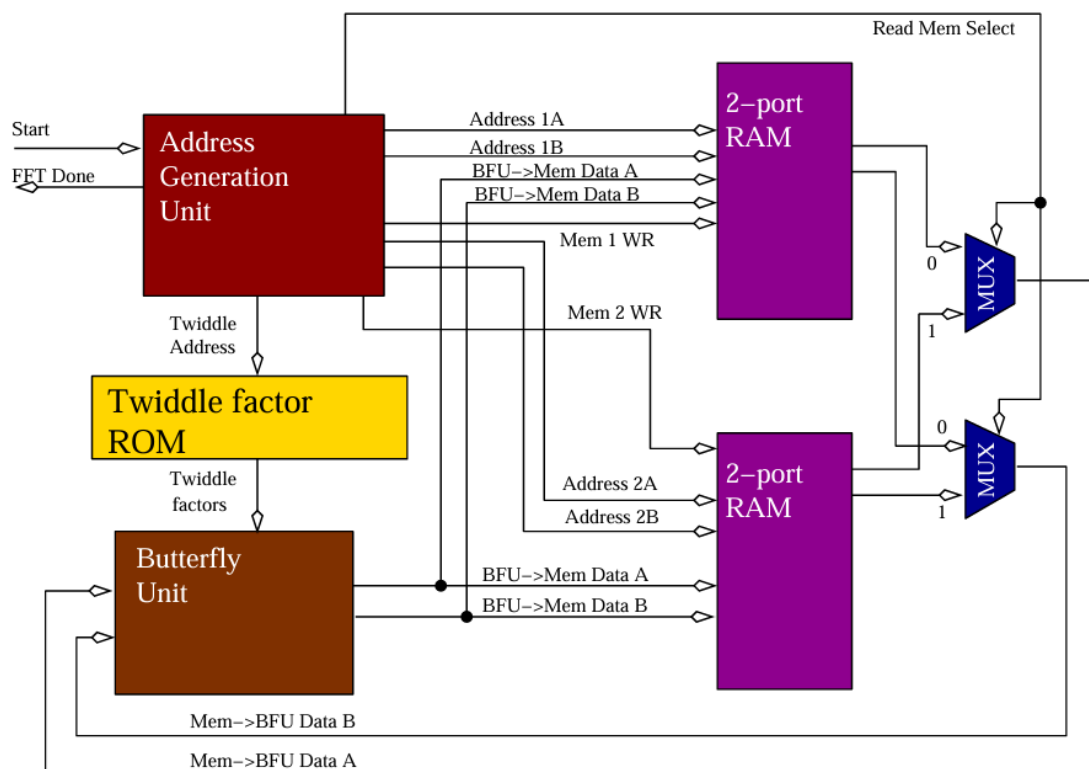
- **32-Point Decimation-in-Time (DIT) Butterfly Structure:** The design utilizes a recursive DIT butterfly approach to split the FFT calculation into manageable stages, enhancing processing speed.
- **Fixed-Point Arithmetic for Efficiency:** The use of fixed-point representation reduces resource demands, making the design both memory and computation efficient.
- **Pipeline Architecture:** The design incorporates pipelining to allow multiple stages of the FFT to execute simultaneously, improving throughput.
- **Parallel Processing:** The FFT implementation supports parallel computation within stages, ensuring faster completion of operations.
- **Scalable and Modular Design:** The structure of the 32-point FFT can be easily modified or extended to accommodate other FFT sizes if needed.
- **Simulation and Testing Suite:** The project includes a comprehensive simulation setup to verify accuracy and performance metrics, ensuring reliable and predictable results.

CHAPTER 2

2.1 BLOCK DIAGRAM

The four main features of the basic block diagram (given below) are

- Butterfly Unit
- Address Generation Unit
- Twiddle Factor ROM
- Dual Port RAM



Data Flow

- The Address Generation Unit is central to controlling the flow of data, generating the necessary addresses to fetch data points and twiddle factors.
- The Butterfly Unit performs the core computations, and the two-port RAMs store the data at various stages of the FFT.

- The Twiddle Factor ROM provides the necessary twiddle factors for the calculations, and the MUX helps in selecting the correct data from memory during the read operations.

2.2 Butterfly Unit (BFU)

- The BFU is the core computational unit in the FFT process.
- It takes two complex inputs (from the two-port RAMs) and a twiddle factor from the ROM, performs complex multiplication, and generates two complex outputs.
- The butterfly computation involves both addition and subtraction operations, combined with the twiddle factors, to calculate the FFT for the input data pairs.
- BFU->Mem Data A and BFU->Mem Data B are the output data from the BFU, which are written back to the two-port RAMs for storage in the next stage of the computation.

$$Y[i] = X[i] + W \times X[j]$$

$$Y[j] = X[i] - W \times X[j]$$

Here:

$X[i]$ and $X[j]$ are the inputs to the butterfly.

W is the twiddle factor.

$Y[i]$ and $Y[j]$ are the outputs after the butterfly operation.

Butterfly Computation Formula

Verilog Code:

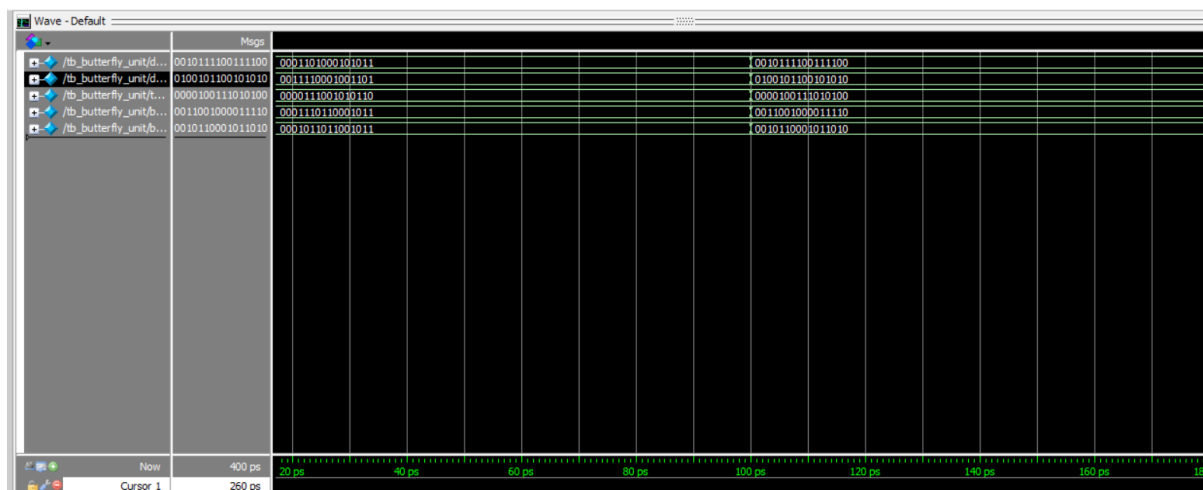
```

module butterfly_unit(
    input [15:0] data_A, data_B, twiddle_factor,
    output reg [15:0] bf_A_out, bf_B_out
);
    wire signed [31:0] mult_out;
    assign mult_out = data_B * twiddle_factor;

    always @(*) begin
        bf_A_out = data_A + mult_out[31:16];
        bf_B_out = data_A - mult_out[31:16];
    end
endmodule

```

Output:



2.3 Address Generation Unit (AGU)

- The AGU is responsible for generating the addresses needed to read and write data in the two-port RAMs and to access the twiddle factors from the ROM.
- It ensures that each stage of the FFT computation has the correct memory addresses for data input to the Butterfly Unit (BFU).
- The AGU typically increments addresses based on the current FFT stage and ensures that data is accessed in the correct sequence for the butterfly operations.
- It outputs addresses such as Address 1A, Address 1B, Address 2A, and Address 2B, which are fed to the two-port RAMs for data retrieval and storage.

Verilog Code:

```

module address_gen_unit(
    input clk, reset,
    input start,
    output reg [4:0] address_A, address_B, // 5-bit for 32 points
    output reg fft_done
);
    reg [4:0] counter;

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            counter <= 0;
            fft_done <= 0;
        end else if (start) begin
            if (counter < 31) begin

```

```

        address_A <= counter;

        address_B <= counter + 1;

        counter <= counter + 2;

    end else begin

        fft_done <= 1;

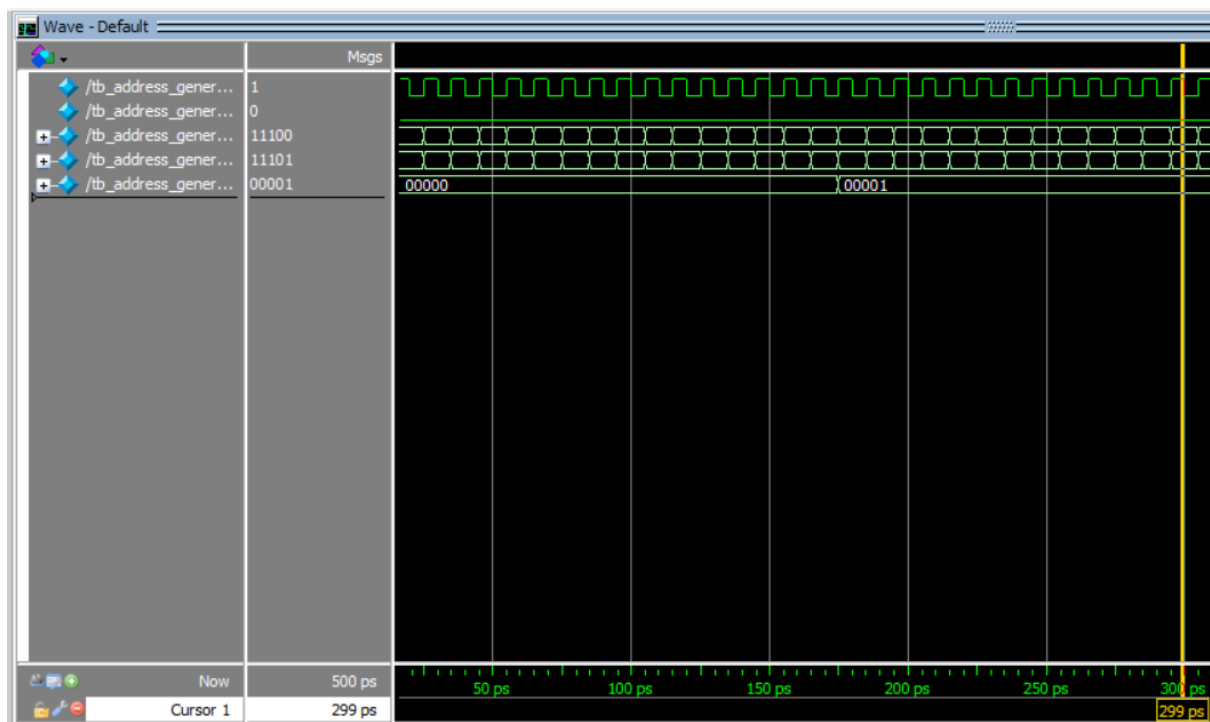
    end

end

endmodule

```

Output:



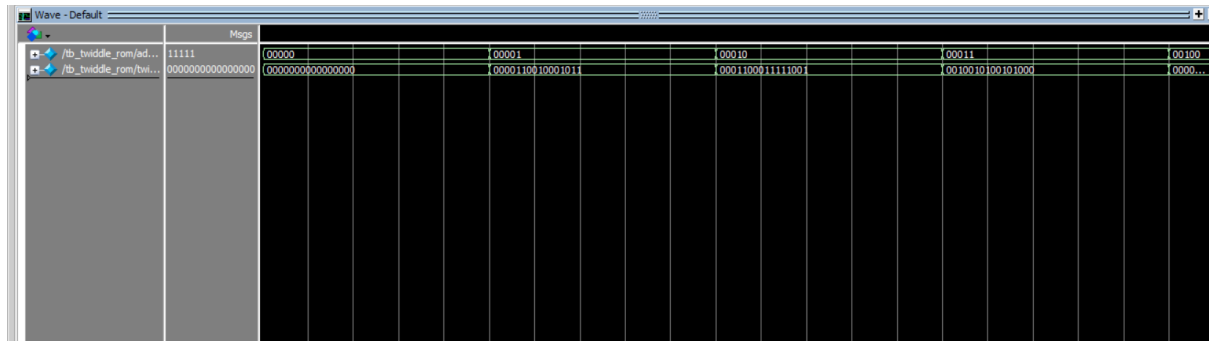
2.4 Twiddle Factor ROM

- This is a Read-Only Memory that stores the precomputed twiddle factors used in each stage of the FFT.
- Twiddle factors are complex exponential values needed to rotate the data points correctly during the butterfly computation.
- The AGU generates a Twiddle Address that selects the specific twiddle factor to be used in each computation stage, based on the FFT algorithm.
- These factors are fed directly into the Butterfly Unit.

Verilog Code:

```
module twiddle_rom(
    input [4:0] address,
    output reg [15:0] twiddle_factor
);
    always @(address) begin
        case (address)
            5'b00000: twiddle_factor = 16'h0000;
            5'b00001: twiddle_factor = 16'h0C8B;
            5'b00010: twiddle_factor = 16'h18F9;
            5'b00011: twiddle_factor = 16'h2528;
            // Add remaining twiddle factors for 32-point FFT
            default: twiddle_factor = 16'h0000;
        endcase
    end
endmodule
```

Output:



2.5 Dual Port RAM

- These are dual-port memory blocks that hold the FFT data at various stages of computation.
- The two-port design allows for simultaneous read and write operations, which speeds up the FFT process by allowing parallel data access.
- Each RAM block can read two addresses (e.g., Address 1A and Address 1B for the first RAM) and write two results simultaneously.
- The output from the BPU (processed data) is stored back in these RAMs, updating the FFT computation data for the next stage.

Verilog Code:

```
module dual_port_ram (
    input clk,
    input [4:0] addr_a, addr_b,
    input [15:0] data_in_a, data_in_b,
    input we_a, we_b,
    output reg [15:0] data_out_a, data_out_b
);
```

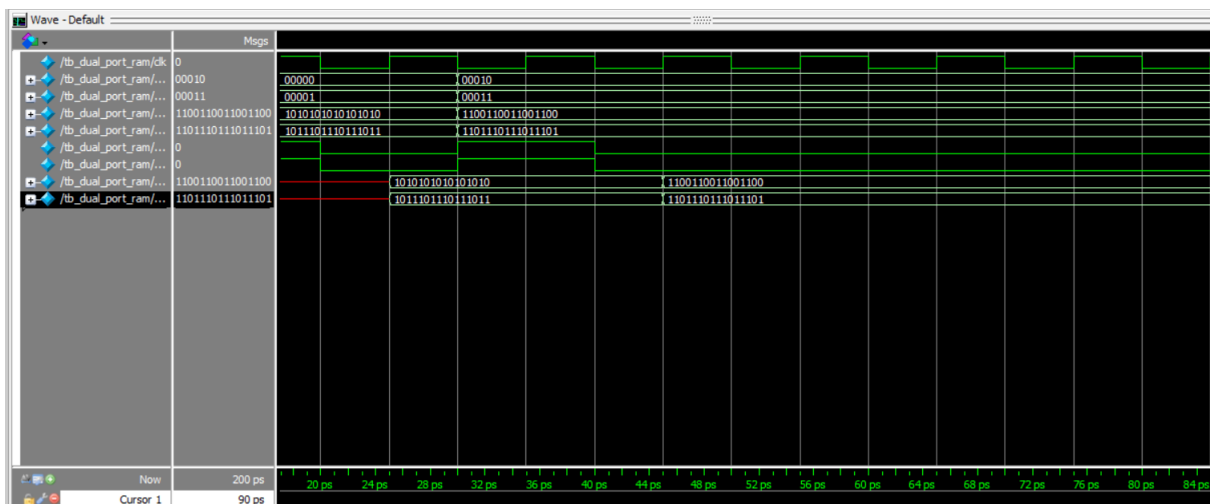
```
reg [15:0] ram [0:31];

always @(posedge clk) begin
    if (we_a) ram[addr_a] <= data_in_a;
    else data_out_a <= ram[addr_a];

    if (we_b) ram[addr_b] <= data_in_b;
    else data_out_b <= ram[addr_b];
end

endmodule
```

Output:



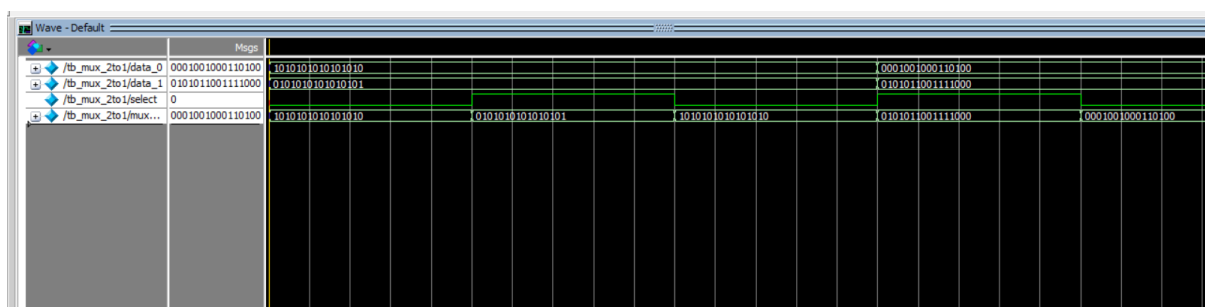
2.6 MUX

- These MUX units handle the selection of data from the RAM blocks.
- Read Mem Select determines which memory addresses or data should be accessed and sent to the output or next computation stage.
- The MUX units streamline the data flow and ensure that the correct results are used at each stage of the computation.

Verilog Code:

```
module mux_2to1 (
    input [15:0] data_0, data_1,
    input select,
    output reg [15:0] mux_out
);
    always @(*) begin
        mux_out = select ? data_1 : data_0;
    end
endmodule
```

Output:



CHAPTER 3

8 – POINT FAST FOURIER TRANSFORM IN VERILOG

3.1 Verilog Code:

```
//31:24 -> Re(INT)
//23:16 -> Re(FRAC)
//15:8 -> Im(INT)
//7:0 -> Im(FRAC)

//numbers upto +_ 32

module dit_fft (
    input [31:0] xn0, xn1, xn2, xn3, xn4, xn5, xn6, xn7,
    output [31:0] xk0, xk1, xk2, xk3, xk4, xk5, xk6, xk7
);
    integer W_0 = 32'h0100_0000; //1
    integer W_1 = 32'h00b5_ff4b; //0.707-0.707j
    integer W_2 = 32'h0000_ff00; //-j
    integer W_3 = 32'hff4b_ff4b; //-0.707-0.707j

    wire [31:0] s10, s11, s12, s13, s14, s15, s16, s17;
    wire [31:0] s20, s21, s22, s23, s24, s25, s26, s27;

    dit_mul m1(xn0, xn4, W_0, s10, s11);
    dit_mul m2(xn2, xn6, W_0, s12, s13);
    dit_mul m3(xn1, xn5, W_0, s14, s15);
    dit_mul m4(xn3, xn7, W_0, s16, s17);

    dit_mul m5(s10, s12, W_0, s20, s22);
```

```

    dit_mul m6(s11, s13, W_2, s21, s23);
    dit_mul m7(s14, s16, W_0, s24, s26);
    dit_mul m8(s15, s17, W_2, s25, s27);

    dit_mul m9(s20, s24, W_0, xk0, xk4);
    dit_mul m10(s21, s25, W_1, xk1, xk5);
    dit_mul m11(s22, s26, W_2, xk2, xk6);
    dit_mul m12(s23, s27, W_3, xk3, xk7);
endmodule

module dit_mul (
    input [31:0] in_up, in_down, twiddle,
    output [31:0] out_up, out_down
);
    wire [31:0] temp;
    comp_mul mul(.a(in_down), .b(twiddle), .y(temp));
    comp_add add(.a(in_up), .b(temp), .y(out_up));
    comp_sub sub(.a(in_up), .b(temp), .y(out_down));
endmodule

module comp_add (
    input [31:0] a, b,
    output [31:0] y
);
    wire signed [15:0] re_A = a[31:16];
    wire signed [15:0] im_A = a[15:0];
    wire signed [15:0] re_B = b[31:16];
    wire signed [15:0] im_B = b[15:0];
    assign y = {(re_A + re_B), (im_A + im_B)};
endmodule

module comp_sub (
    input [31:0] a, b,
    output [31:0] y
);
    wire signed [15:0] re_A = a[31:16];

```

```

    wire signed [15:0] im_A = a[15:0];
    wire signed [15:0] re_B = b[31:16];
    wire signed [15:0] im_B = b[15:0];
    assign y = {(re_A - re_B), (im_A - im_B)};
endmodule

//(a + ib) (c + id) = (ac - bd) + i(ad + bc)
module comp_mul (
    input [31:0] a, b,
    output [31:0] y
);
    wire signed [15:0] re_A = a[31:16];
    wire signed [15:0] im_A = a[15:0];
    wire signed [15:0] re_B = b[31:16];
    wire signed [15:0] im_B = b[15:0];
    wire [31:0] ac = re_A * re_B;
    wire [31:0] bd = im_A * im_B;
    wire [31:0] ad = re_A * im_B;
    wire [31:0] bc = re_B * im_A;
    assign y = {($signed(ac[23:8])-$
$signed(bd[23:8])),($signed(ad[23:8])+$signed(bc[23:8]))};
endmodule

```

Output:

	Msgs
/dit_fft_tb/xn0	00011101000111...
/dit_fft_tb/xn1	01100011101011...
/dit_fft_tb/xn2	10100111110101...
/dit_fft_tb/xn3	01110001110101...
/dit_fft_tb/xn4	00000001111110...
/dit_fft_tb/xn5	00101111010100...
/dit_fft_tb/xn6	00001111000011...
/dit_fft_tb/xn7	11111001100111...
/dit_fft_tb/xk0	11010100011011...
/dit_fft_tb/xk1	00000111010011...
/dit_fft_tb/xk2	00000111000000...
/dit_fft_tb/xk3	10011101001110...
/dit_fft_tb/xk4	11010111100001...
/dit_fft_tb/xk5	11111001000001...
/dit_fft_tb/xk6	11001000111010...
/dit_fft_tb/xk7	11001111000000...

32 – POINT FAST FOURIER TRANSFORM IN VERILOG

3.2 Verilog Code

```

module dit_fft (
    input [31:0] xn0, xn1, xn2, xn3, xn4, xn5, xn6, xn7,
    input [31:0] xn8, xn9, xn10, xn11, xn12, xn13, xn14, xn15,
    input [31:0] xn16, xn17, xn18, xn19, xn20, xn21, xn22, xn23,
    input [31:0] xn24, xn25, xn26, xn27, xn28, xn29, xn30, xn31,
    output [31:0] xk0, xk1, xk2, xk3, xk4, xk5, xk6, xk7,
    output [31:0] xk8, xk9, xk10, xk11, xk12, xk13, xk14, xk15,
    output [31:0] xk16, xk17, xk18, xk19, xk20, xk21, xk22, xk23,
    output [31:0] xk24, xk25, xk26, xk27, xk28, xk29, xk30, xk31
);
    // Twiddle factor initialization
    integer W_0 = 32'h0100_0000; // 1
    integer W_1 = 32'h00b5_ff4b; // 0.707 - 0.707j
    integer W_2 = 32'h0000_ff00; // -j
    integer W_3 = 32'hff4b_ff4b; // -0.707 - 0.707j

    wire [31:0] s10, s11, s12, s13, s14, s15, s16, s17;
    wire [31:0] s20, s21, s22, s23, s24, s25, s26, s27;
    wire [31:0] s30, s31, s32, s33, s34, s35, s36, s37;
    wire [31:0] s40, s41, s42, s43, s44, s45, s46, s47;

    // Stage 1: 8-point FFT computations for each chunk
    dit_mul m1(xn0, xn4, W_0, s10, s11);
    dit_mul m2(xn1, xn5, W_0, s12, s13);
    dit_mul m3(xn2, xn6, W_0, s14, s15);
    dit_mul m4(xn3, xn7, W_0, s16, s17);

    dit_mul m5(xn8, xn12, W_0, s20, s21);
    dit_mul m6(xn9, xn13, W_0, s22, s23);
    dit_mul m7(xn10, xn14, W_0, s24, s25);
    dit_mul m8(xn11, xn15, W_0, s26, s27);

    dit_mul m9(xn16, xn20, W_0, s30, s31);
    dit_mul m10(xn17, xn21, W_0, s32, s33);
    dit_mul m11(xn18, xn22, W_0, s34, s35);
    dit_mul m12(xn19, xn23, W_0, s36, s37);

    dit_mul m13(xn24, xn28, W_0, s40, s41);
    dit_mul m14(xn25, xn29, W_0, s42, s43);
    dit_mul m15(xn26, xn30, W_0, s44, s45);
    dit_mul m16(xn27, xn31, W_0, s46, s47);

    // Stage 2: Further FFT calculations (combining the smaller FFTs)

```

```

// Example: Combine the results with twiddle factors for each pair
dit_mul m17(s10, s12, W_0, xk0, xk8);
dit_mul m18(s11, s13, W_1, xk1, xk9);
dit_mul m19(s14, s16, W_0, xk2, xk10);
dit_mul m20(s15, s17, W_1, xk3, xk11);

dit_mul m21(s20, s22, W_0, xk4, xk12);
dit_mul m22(s21, s23, W_1, xk5, xk13);
dit_mul m23(s24, s26, W_0, xk6, xk14);
dit_mul m24(s25, s27, W_1, xk7, xk15);

dit_mul m25(s30, s32, W_0, xk16, xk24);
dit_mul m26(s31, s33, W_1, xk17, xk25);
dit_mul m27(s34, s36, W_0, xk18, xk26);
dit_mul m28(s35, s37, W_1, xk19, xk27);

dit_mul m29(s40, s42, W_0, xk20, xk28);
dit_mul m30(s41, s43, W_1, xk21, xk29);
dit_mul m31(s44, s46, W_0, xk22, xk30);
dit_mul m32(s45, s47, W_1, xk23, xk31);

endmodule

module dit_mul (
    input [31:0] in_up, in_down, twiddle,
    output [31:0] out_up, out_down
);
    wire [31:0] temp;
    comp_mul mul(.a(in_down), .b(twiddle), .y(temp));
    comp_add add(.a(in_up), .b(temp), .y(out_up));
    comp_sub sub(.a(in_up), .b(temp), .y(out_down));
endmodule

module comp_add (
    input [31:0] a, b,
    output [31:0] y
);
    wire signed [15:0] re_A = a[31:16];
    wire signed [15:0] im_A = a[15:0];
    wire signed [15:0] re_B = b[31:16];
    wire signed [15:0] im_B = b[15:0];
    assign y = {(re_A + re_B), (im_A + im_B)};
endmodule

module comp_sub (
    input [31:0] a, b,
    output [31:0] y
);
    wire signed [15:0] re_A = a[31:16];
    wire signed [15:0] im_A = a[15:0];
    wire signed [15:0] re_B = b[31:16];
    wire signed [15:0] im_B = b[15:0];
    assign y = {(re_A - re_B), (im_A - im_B)};
endmodule

```

```

module comp_mul (
    input [31:0] a, b,
    output [31:0] y
);
    wire signed [15:0] re_A = a[31:16];
    wire signed [15:0] im_A = a[15:0];
    wire signed [15:0] re_B = b[31:16];
    wire signed [15:0] im_B = b[15:0];
    wire [31:0] ac = re_A * re_B;
    wire [31:0] bd = im_A * im_B;
    wire [31:0] ad = re_A * im_B;
    wire [31:0] bc = re_B * im_A;
    assign y = {($signed(ac[23:8]))-
$signed(bd[23:8])),($signed(ad[23:8])+$signed(bc[23:8]))};
endmodule

```

Testbench:

```

module tb_dit_fft;

    // Inputs
    reg [31:0] xn0, xn1, xn2, xn3, xn4, xn5, xn6, xn7;
    reg [31:0] xn8, xn9, xn10, xn11, xn12, xn13, xn14, xn15;
    reg [31:0] xn16, xn17, xn18, xn19, xn20, xn21, xn22, xn23;
    reg [31:0] xn24, xn25, xn26, xn27, xn28, xn29, xn30, xn31;

    // Outputs
    wire [31:0] xk0, xk1, xk2, xk3, xk4, xk5, xk6, xk7;
    wire [31:0] xk8, xk9, xk10, xk11, xk12, xk13, xk14, xk15;
    wire [31:0] xk16, xk17, xk18, xk19, xk20, xk21, xk22, xk23;
    wire [31:0] xk24, xk25, xk26, xk27, xk28, xk29, xk30, xk31;

    // Instantiate the Unit Under Test (UUT)
    dit_fft uut (
        .xn0(xn0), .xn1(xn1), .xn2(xn2), .xn3(xn3), .xn4(xn4), .xn5(xn5),
        .xn6(xn6), .xn7(xn7), .xn8(xn8), .xn9(xn9), .xn10(xn10),
        .xn11(xn11),
        .xn12(xn12), .xn13(xn13), .xn14(xn14), .xn15(xn15), .xn16(xn16),
        .xn17(xn17), .xn18(xn18), .xn19(xn19), .xn20(xn20), .xn21(xn21),
        .xn22(xn22), .xn23(xn23), .xn24(xn24), .xn25(xn25), .xn26(xn26),
        .xn27(xn27), .xn28(xn28), .xn29(xn29), .xn30(xn30), .xn31(xn31),
        .xk0(xk0), .xk1(xk1), .xk2(xk2), .xk3(xk3), .xk4(xk4), .xk5(xk5),
        .xk6(xk6), .xk7(xk7), .xk8(xk8), .xk9(xk9), .xk10(xk10),
        .xk11(xk11),
        .xk12(xk12), .xk13(xk13), .xk14(xk14), .xk15(xk15), .xk16(xk16),
        .xk17(xk17), .xk18(xk18), .xk19(xk19), .xk20(xk20), .xk21(xk21),
        .xk22(xk22), .xk23(xk23), .xk24(xk24), .xk25(xk25), .xk26(xk26),
        .xk27(xk27), .xk28(xk28), .xk29(xk29), .xk30(xk30), .xk31(xk31)
    );

    // Stimulus

```

```

initial begin
    // Initialize inputs to some test values
    xn0 = 32'h01000000; // Real part 1, Imaginary part 0
    xn1 = 32'h02000000; // Real part 2, Imaginary part 0
    xn2 = 32'h03000000; // Real part 3, Imaginary part 0
    xn3 = 32'h04000000; // Real part 4, Imaginary part 0
    xn4 = 32'h05000000; // Real part 5, Imaginary part 0
    xn5 = 32'h06000000; // Real part 6, Imaginary part 0
    xn6 = 32'h07000000; // Real part 7, Imaginary part 0
    xn7 = 32'h08000000; // Real part 8, Imaginary part 0
    xn8 = 32'h09000000; // Real part 9, Imaginary part 0
    xn9 = 32'h0A000000; // Real part 10, Imaginary part 0
    xn10 = 32'h0B000000; // Real part 11, Imaginary part 0
    xn11 = 32'h0C000000; // Real part 12, Imaginary part 0
    xn12 = 32'h0D000000; // Real part 13, Imaginary part 0
    xn13 = 32'h0E000000; // Real part 14, Imaginary part 0
    xn14 = 32'h0F000000; // Real part 15, Imaginary part 0
    xn15 = 32'h10000000; // Real part 16, Imaginary part 0
    xn16 = 32'h11000000; // Real part 17, Imaginary part 0
    xn17 = 32'h12000000; // Real part 18, Imaginary part 0
    xn18 = 32'h13000000; // Real part 19, Imaginary part 0
    xn19 = 32'h14000000; // Real part 20, Imaginary part 0
    xn20 = 32'h15000000; // Real part 21, Imaginary part 0
    xn21 = 32'h16000000; // Real part 22, Imaginary part 0
    xn22 = 32'h17000000; // Real part 23, Imaginary part 0
    xn23 = 32'h18000000; // Real part 24, Imaginary part 0
    xn24 = 32'h19000000; // Real part 25, Imaginary part 0
    xn25 = 32'h1A000000; // Real part 26, Imaginary part 0
    xn26 = 32'h1B000000; // Real part 27, Imaginary part 0
    xn27 = 32'h1C000000; // Real part 28, Imaginary part 0
    xn28 = 32'h1D000000; // Real part 29, Imaginary part 0
    xn29 = 32'h1E000000; // Real part 30, Imaginary part 0
    xn30 = 32'h1F000000; // Real part 31, Imaginary part 0
    xn31 = 32'h20000000; // Real part 32, Imaginary part 0

    // Wait for some time to observe the output
    #10;

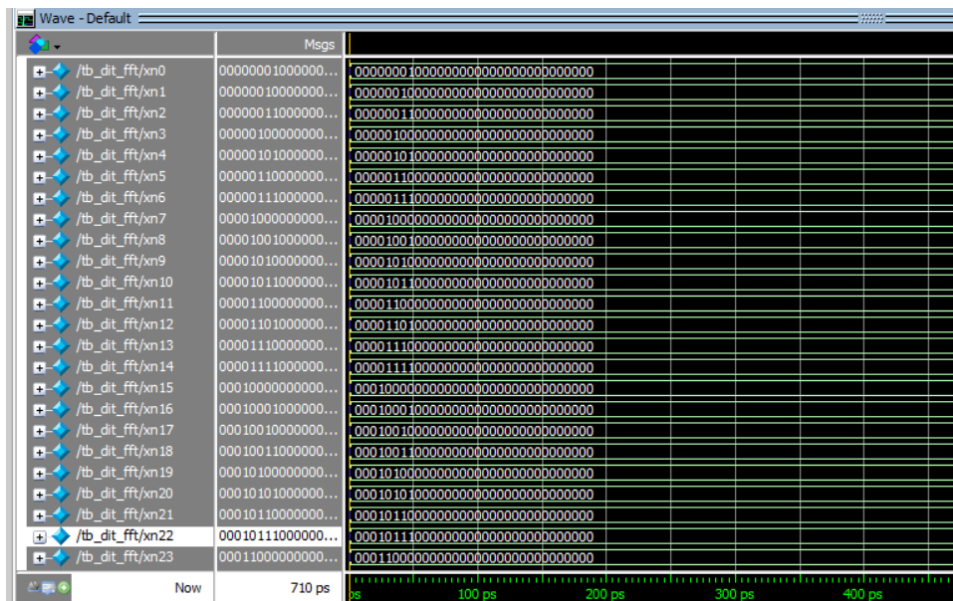
    // End the simulation after a set period
    $finish;
end

// Monitor the outputs to observe the result
initial begin
    $monitor("Time: %t | xk0: %h | xk1: %h | xk2: %h | xk3: %h | xk4: %h | xk5: %h | xk6: %h | xk7: %h",
        $time, xk0, xk1, xk2, xk3, xk4, xk5, xk6, xk7);
end

endmodule

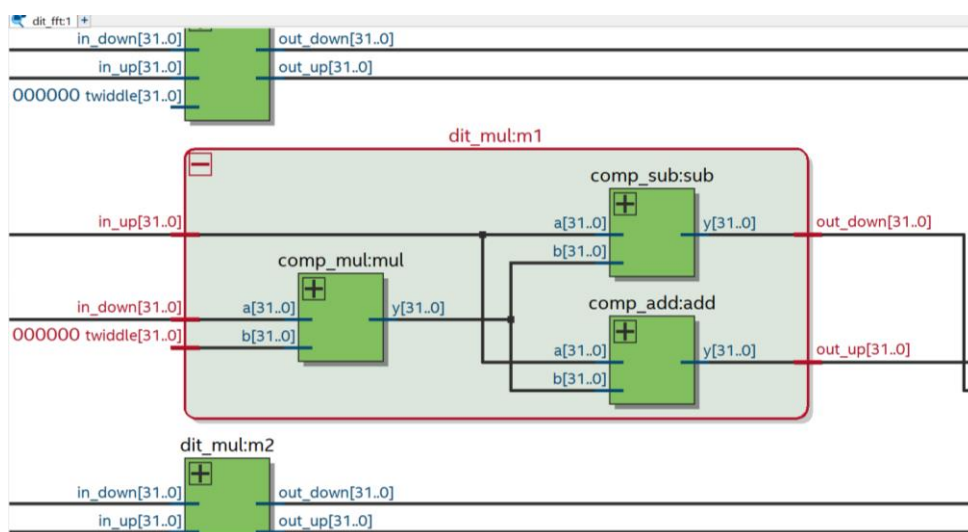
```

3.2 Result

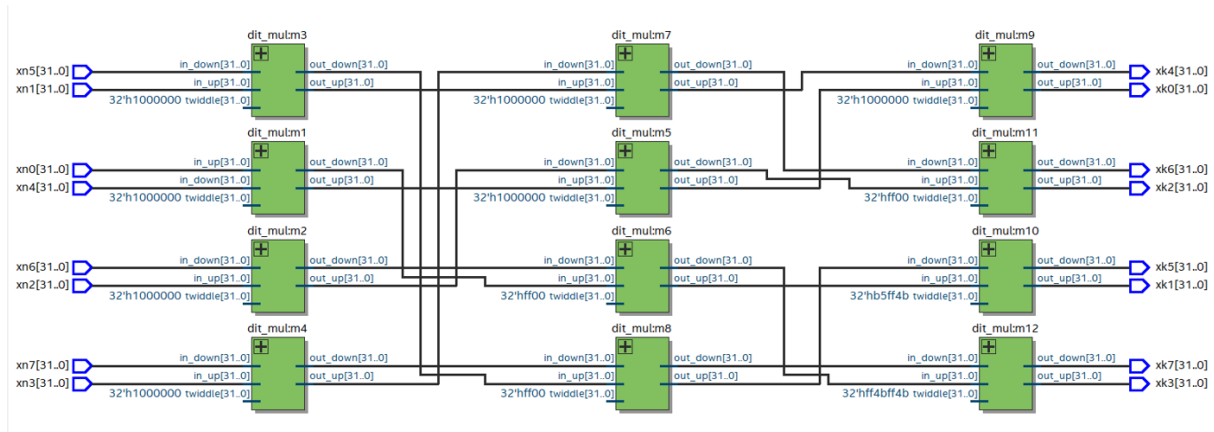


3.3 RTL:

DIT Logic Block



DIT Module



3.4 Inferences

Efficient Data Management:

- The two-port RAM architecture allows for simultaneous read and write operations, which enhances the throughput of the FFT computation. This design is likely chosen to minimize data access time and maximize the speed of data processing, a crucial requirement for FFT algorithms.

Parallel Processing Capability:

- The Address Generation Unit (AGU) provides addresses for two data points simultaneously, allowing the FFT computation to process multiple data elements in parallel. This approach reduces the total time required to complete the FFT.

Simplified Control Flow:

- The presence of control signals like Start and FFT Done indicates a straightforward start-to-finish control flow. The Start signal triggers the computation, and FFT Done signals its completion, simplifying integration with other digital components or systems..

Modular Design:

- The distinct components (AGU, Twiddle Factor ROM, BFU, RAMs) suggest a modular approach to the FFT implementation. Each module performs a specific function, making it easier to debug, optimize, and potentially reuse parts in other designs.

Data Flow Control with MUXes:

- The MUX units control which data outputs are selected from the RAM blocks, allowing for flexibility in reading and writing data in the proper sequence. This control mechanism could also support pipelining, where multiple stages of FFT can run in sequence without waiting for the entire computation to finish.

Potential for Scalability:

- This modular design with dedicated memory and a centralized processing unit (BFU) makes it scalable for different FFT sizes. By adjusting the RAM size, twiddle factor ROM, and addressing scheme, this design could be adapted for larger FFT computations with minimal changes.

Optimized for Hardware Implementation:

- The architecture shown is well-suited for FPGA or ASIC implementations, as it leverages hardware-specific advantages such as parallel processing, memory efficiency, and modularity. This design likely achieves high performance and low latency, making it ideal for real-time signal processing applications.

CHAPTER 4

CONCLUSION

This project successfully showcases a practical and efficient way to implement the Fast Fourier Transform (FFT) in hardware. By carefully designing each component, including the Address Generation Unit, Twiddle Factor ROM, Butterfly Unit, and dual-port RAMs, the system achieves high-speed data processing, which is essential for real-time applications.

The modular structure of each unit adds flexibility, allowing the design to be scaled up for larger FFT sizes or adapted to different hardware platforms, like FPGAs. With precomputed twiddle factors stored in ROM and the use of dual-port RAM, data flows smoothly, minimizing interruptions and reducing processing delays. This makes the design well-suited for applications that need quick, accurate frequency analysis, such as digital signal processing in communication systems or radar.

Overall, this project demonstrates a strong, practical approach to FFT computation in hardware, balancing speed, flexibility, and efficient resource use. It provides a solid foundation for future improvements and can be adapted for even more complex, real-time signal processing applications.

CHAPTER 5

REFERENCES

- [1] <https://web.mit.edu/6.111/www/f2017/handouts/FFTtutorial121102.pdf>
- [2] <https://www.iosrjournals.org/iosr-jece/papers/Vol9-Issue1/Version-2/F09124043.pdf>
- [3] <https://ece.anits.edu.in/Project%20Reports%202020-21/Sec-B/B-10.pdf>
- [4] <http://www.ijvdc.org/uploads/251643IJVDCS5756-120.pdf>

BIODATA

Name : S Shanmati

Mobile Number : 8825572236

E-mail : shanmati.s2022@vitstudent.ac.in

Permanent Address: D-219, SIS queenstown, Guduvanchery, Chennai-603202



Name : Sanjana A

Mobile Number : 9444421085

E-mail : sanjana.a2022@vitstudent.ac.in

Permanent Address: Unitech Palm Villa: 56, Nallambakkam, Chennai - 600 127



Name : V.B.Varshini

Mobile Number : 8838057049

E-mail : varshini.vb2022@vitstudent.ac.in

Permanent Address: 45/21 Sri abirami apartments,new street,
Mylapore,chennai -04